# SPCA: Sequence-Position Compression with Structured Position Maps

**Shubham**

*Technical report and prototype evaluation*

## Abstract

We describe SPCA (Sequence-Position Compression Algorithm), a lossless compression scheme that replaces repeated byte sequences with dictionary tokens and then compresses the *structure of the token occurrence positions* rather than storing each position independently. Positions are represented as a first position, a count, and either a constant gap, a repeating gap pattern, or a list of variable-length integer gaps, whichever is cheapest. A cost-based selector compares several complete encodings, including direct zlib, and emits the smallest. We implement a prototype with a fixed binary format and evaluate it on 22 files covering text, source code, structured records, periodic data, random data, executables, and images (BMP, PNG, JPEG). All files round-trip exactly. SPCA is substantially smaller than zlib on strictly periodic or repeated inputs (for example 32 bytes versus 120 for a 100,000-byte constant file, and 321 versus 713 for a sawtooth sequence), and on that sawtooth input it also beats LZMA. On text, code, CSV, and executables the selector falls back to zlib, and bzip2 and LZMA are smaller than SPCA on most real-world data. We discuss why, and identify the extensions most likely to change this result. The code and benchmark are released with this report.

## 1. Introduction

Most general-purpose compressors exploit repetition implicitly. LZ77-family methods encode a repeat as a (distance, length) pair, and entropy coders then compress those values. SPCA takes a different organizing view: select a set of repeated sequences, give each a token, and treat the occurrences of each token as a *position set* whose regularity can be exploited directly. If a sequence occurs at positions 0, 4, 8, 12, ..., the whole set can be described by three numbers.

The motivating hypothesis is that this representation can expose regularity that a stream of independent match references encodes less efficiently. This report tests that hypothesis with a prototype. It does not claim a new state of the art. The contribution is (i)

a precisely specified format, (ii) an open implementation, and (iii) an honest measurement of where the idea helps and where it does not.

## 2. Related work

SPCA sits near several established ideas. LZ77 and LZ78 (Ziv and Lempel, 1977, 1978) replace repeats with references to earlier data or a growing dictionary. DEFLATE (Deutsch, 1996), used by zlib, combines LZ77 with Huffman coding (Huffman, 1952). Grammar-based methods such as Sequitur (Nevill-Manning and Witten, 1997) and Re-Pair (Larsson and Moffat, 2000) build a hierarchical dictionary of repeated phrases, which is closest to SPCA's tokenization, though they encode the reduced sequence rather than separate position maps. The Burrows-Wheeler transform (Burrows and Wheeler, 1994), used by bzip2, reorders data to cluster similar contexts, and LZMA combines long-range LZ matching with context-modelled range coding. Run-length encoding and delta coding capture constant-gap structure in special cases. SPCA's position maps generalize the run-length idea to occurrences of arbitrary tokens, with periodic gap patterns as a further extension.

## 3. Method

### 3.1 Overview

Given an input byte string x of length n, SPCA performs the following steps.

1. Discover repeated sequences and select a dictionary by net estimated saving.
2. Parse x into a sequence of N segments, each either a dictionary token or a single literal byte.
3. For each token, record the segment indices at which it occurs and encode them as a position map.
4. Assemble several candidate encodings, compute the exact size of each, and emit the smallest.

### 3.2 Repeated-sequence discovery

For each candidate length L in {64, 32, 16, 8, 6, 4}, all substrings of that length are counted using a hash map. A substring occurring c >= 2 times has estimated saving

```
save = c * L - (L + 2) - 2c
```

where L + 2 approximates the dictionary entry cost and 2c approximates the occurrence cost. Candidates with positive saving are sorted by decreasing saving, with ties broken by length and byte value so the order is deterministic. Selection is greedy: for each candidate, occurrences that do not overlap bytes already claimed are accepted. After overlap resolution the true saving, len(accepted) * (L - 2) - (L + 2), is rechecked, and the candidate is rejected if it is no longer positive. The dictionary is capped at 2,048 entries. Frequency alone is not used, because a frequent short sequence can cost more to reference than it saves.

## 3.3 Tokenization and segment indexing

The dictionary is ordered by first byte position. The input is scanned left to right: at a byte offset where a selected sequence begins, a token segment is emitted and the scan advances by the sequence length; otherwise a literal segment is emitted and the byte is appended to a separate literal stream. The result is N segments. Positions in the token-position map are *segment indices*, not byte offsets, which keeps the gaps small and makes the map independent of token lengths.

## 3.4 Position maps

For a token with sorted segment indices $p_1 < p_2 < \ldots < p_k$, define the gap sequence $g_1 = p_1$ and $g_i = p_i - p_{(i-1)}$ for $i > 1$. The positions are exactly recoverable from the gaps. Three encodings are considered and the shortest is used:

- **Constant gap (mode 0).** Used when $k >= 3$ and $g_2 = \ldots = g_k$. Stores first position, count, and the common gap.
- **Periodic gap (mode 1).** Used when $k >= 3$ and the sequence $g_2, \ldots, g_k$ is a pattern of period $p$ ($2 <= p <= 16$) repeated an exact whole number of times. Stores first position, count, the period, and one copy of the pattern.
- **Irregular (mode 2).** Stores the count and every gap as a variable-length integer (7 data bits per byte, high bit as continuation).

Each map begins with a one-byte mode tag. The full cost of the representation, including tag and parameters, is compared, so a pattern mode is only used when it is actually smaller.

## 3.5 Candidate encodings and selection

The following complete encodings are built and sized:

1. **stored**: the input unchanged.

2. **zlib-direct**: the input compressed with zlib at level 9.
3. **spca-raw**: header, dictionary, position maps, and literals with no further compression.
4. **spca+zlib**: the spca-raw body compressed with zlib.
5. **spca-split**: the dictionary and position maps compressed together with zlib, and the literal stream compressed separately.

The total size is S = S_header + S_dictionary + S_maps + S_payload, with a leading one-byte method tag. The smallest candidate is emitted. Because direct zlib and stored are always candidates, SPCA is never larger than zlib by more than one byte, and never expands data by more than one byte. This is a property of the selector, not evidence of the quality of the transform, and results are interpreted with that in mind.

### 3.6 Container format and decoding

SPCA methods carry a header with the original length (varint) and a CRC-32 of the original data. The SPCA body consists of N and the dictionary size D (varints), D dictionary entries (length varint plus bytes), D position maps in dictionary order, and the remaining literal bytes. The literal count is implied (N minus the total token count). Decoding reads the body, expands each position map into segment indices, fills those segments with the corresponding dictionary entries, fills the remaining segments with literals in order, and verifies the length and CRC-32. Token order, overlap handling, and integer encoding are fully determined by the format, so any conforming decoder reconstructs the same bytes.

## 4. Experimental setup

### 4.1 Corpus

The corpus has 22 files of up to about 200 KB, chosen to cover the categories in the design: text, source code, structured records, periodic data, repeated bytes, random data, already-compressed data, and images.

- **Text and code:** a Python standard-library source file, a C header, and an English license text.
- **Structured records:** a 4,000-row synthetic CSV and a 3,000-object synthetic JSON file with random but low-cardinality fields.

- **Regular data:** 100,000 identical zero bytes; the sequence ABCD repeated 25,000 times; a 6-byte irregular-looking motif (xy, three dots, z) repeated 20,000 times; and a sawtooth sequence i mod 251.
- **Incompressible or precompressed:** 100,000 random bytes, a zlib-compressed text file, and the first 100,000 bytes of an ELF executable (a realistic binary that is not precompressed).
- **Images (256 x 256, RGB):** a flat-colour graphic with text, a smooth gradient, a photo-like smooth field with Gaussian noise, a pure-noise image, and a 16 x 16 tile repeated 16 x 16 times. Each is stored as BMP (uncompressed); several are also stored as PNG, and the photo-like image additionally as JPEG at quality 85.

Synthetic data was generated with fixed random seeds, except the random-byte file, which uses the operating system's entropy source and therefore differs between runs.

### 4.2 Baselines and metrics

Baselines are zlib, bzip2, and LZMA, each at maximum level (zlib 9, bzip2 9, LZMA preset 9) via the Python standard library. The primary metric is compressed size in bytes. Every output was decompressed and compared byte-for-byte with the original. Compression wall-clock time was recorded per file but is not a rigorous timing benchmark (single run, unoptimized pure-Python implementation).

## 5. Results

Table 1 lists compressed sizes in bytes. The SPCA column is the size of the final emitted output, including selector fallback.

| File | Original | zlib | bzip2 | LZMA | SPCA | SPCA mode |
|---|---|---|---|---|---|---|
| repeated_byte | 100,000 | 120 | 47 | 148 | 32 | spca+zlib |
| periodic_abcd | 100,000 | 125 | 51 | 152 | 37 | spca+zlib |
| periodic_irregular | 120,000 | 203 | 55 | 156 | 62 | spca+zlib |
| sawtooth | 100,000 | 713 | 924 | 376 | 321 | spca+zlib |
| tiled_pattern.bmp | 196,662 | 1,896 | 2,063 | 1,356 | 1,450 | spca+zlib |
| json_records | 103,890 | 9,773 | 7,542 | 5,896 | 9,664 | spca-split |
| source_code.py | 101,750 | 21,428 | 18,133 | 19,592 | 21,429 | zlib-direct |
| source_code_c.h | 34,649 | 6,805 | 6,505 | 6,484 | 6,806 | zlib-direct |

| File | Original | zlib | bzip2 | LZMA | SPCA | SPCA mode |
|---|---|---|---|---|---|---|
| english_text | 35,149 | 12,112 | 10,706 | 11,428 | 12,113 | zlib-direct |
| records.csv | 94,788 | 26,102 | 18,756 | 20,568 | 26,103 | zlib-direct |
| executable_elf | 100,000 | 47,617 | 46,202 | 42,248 | 47,618 | zlib-direct |
| graphic_flat.bmp | 196,662 | 1,814 | 1,327 | 964 | 1,815 | zlib-direct |
| gradient.bmp | 196,662 | 139,088 | 18,624 | 16,648 | 139,089 | zlib-direct |
| photolike.bmp | 196,662 | 184,519 | 192,834 | 171,436 | 184,520 | zlib-direct |
| noise.bmp | 196,662 | 196,728 | 197,890 | 196,732 | 196,663 | stored |
| random.bin | 100,000 | 100,041 | 100,831 | 100,064 | 100,001 | stored |
| already_zlib.bin | 12,112 | 12,123 | 12,591 | 12,172 | 12,113 | stored |

Small already-compressed image files (graphic_flat.png, gradient.png, photolike.png, photolike.jpg, tiled_pattern.png) behave like the precompressed case: SPCA is within one byte of the best of zlib and stored, and no method achieves meaningful further reduction. Their full results are in the released `results.json`.

All 22 files decompressed to byte-identical output.

## 5.1 Observations

**Regular data.** On the four regular synthetic files, SPCA is smaller than zlib by factors of roughly 4 (repeated byte), 3.4 (ABCD), 3.3 (irregular motif), and 2.2 (sawtooth). It beats LZMA on three of these (the repeated-byte, ABCD, and sawtooth files) and is within a factor of 1.6 of LZMA on the motif file, where bzip2 is smallest at 55 bytes. On the repeated-byte, ABCD, and motif files bzip2 is the smallest of all methods, so SPCA does not dominate on this category. SPCA's best result relative to every baseline is the sawtooth file (321 versus 376 for LZMA, 713 for zlib, and 924 for bzip2). These inputs are exactly the case the position-map modes target, since a token's occurrences form a constant or periodic gap sequence.

**Tiled image.** On the tiled-pattern BMP, SPCA (1,450) is 24 percent smaller than zlib (1,896) but 7 percent larger than LZMA (1,356). The tile repeats at row-aligned intervals, so the position-map structure helps, but LZMA's long-range matching remains stronger.

**Real-world data.** For text, source code, CSV, and executables the selector chose direct zlib, meaning no SPCA variant was smaller. bzip2 and LZMA are 10 to 28 percent smaller than zlib on these files, so they are also smaller than SPCA. On JSON, spca-split was

marginally smaller than zlib (9,664 versus 9,773, about 1 percent), while LZMA reached 5,896 and bzip2 7,542.

**Smooth images.** On the gradient BMP, SPCA falls back to zlib at 139,089 bytes, whereas bzip2 and LZMA reach 18,624 and 16,648, about eight times smaller. Neighbouring pixels in a gradient differ by small, regular amounts, which is numeric structure rather than exact repetition. The photo-like image is nearly incompressible for all methods because of its additive noise, and the pure-noise image and random bytes are incompressible by any method, as expected.

**Precompressed and incompressible data.** The selector correctly stores incompressible and already-compressed inputs, adding one byte of overhead, whereas zlib, bzip2, and LZMA each expand them by tens to hundreds of bytes.

## 6. Analysis and discussion

**Why position maps help on regular data.** When a token's occurrences are exactly periodic, the entire position set collapses to a few integers, and the result is largely independent of file size. This explains the near-constant sizes on the synthetic regular files. An LZ77 stream also compresses such data well because long overlapping matches are cheap, but it still emits a stream of match tokens that an entropy coder must then compress, whereas SPCA states the regularity explicitly.

**Why it does not help on realistic data.** In text, code, and records, repeated sequences occur at irregular positions, so most position maps fall back to the irregular mode, which is varint gaps. That is comparable to, and in our implementation less well modelled than, the (distance, length) pairs of an entropy-coded LZ stream. The tokenization also removes the local context that adaptive entropy coders and BWT-based methods exploit between adjacent symbols, which likely explains why bzip2 and LZMA remain stronger.

**The selector's role.** Including direct zlib as a candidate guarantees SPCA is never materially worse than zlib. This is useful engineering but is also why the "zlib-direct" rows in Table 1 should not be counted as SPCA successes. The honest summary is that the transformed representations win on strictly regular inputs and are otherwise not competitive with the strongest baselines.

**Metadata overhead.** The dictionary and map costs are real: for spca-raw output on realistic files, the structure (dictionary plus maps) is a significant fraction of the output. The compressed split variant, which compresses structure and literals separately, helps only slightly on JSON and not at all on other realistic files in our tests.

## 7. Limitations

- **Small and partly synthetic corpus.** The corpus has 22 files up to about 200 KB, and much of it is synthetic. The regular-data results in particular are favourable by construction. Claims about held-out real-world files need a larger benchmark such as the Canterbury, Silesia, or Calgary corpora, and a real image set.
- **Single run, no variance estimates.** Sizes are deterministic for fixed inputs, except for the random-byte file; timing was measured once.
- **Prototype implementation.** The code is unoptimized pure Python. Compression time of 0.1 to 0.5 seconds on files up to 200 KB should not be extrapolated to large files, because discovery stores all occurrences of every candidate substring, and memory grows with input size. Block-wise processing, described in the design, is not implemented.
- **Simple parsing.** Selection is greedy and based on an approximate saving model. Optimal parsing, as in modern LZ compressors, could change the results.
- **Back-end coding.** The final stage relies on zlib. A dedicated entropy coder for literals, tokens, and gaps (Huffman, arithmetic, or range coding with context modelling) is not implemented and is the largest source of the gap to LZMA.
- **Image handling.** No image-specific modelling is used; images are treated as byte streams. Row-delta or predictive filters, as used in PNG, would likely help gradients and photographs substantially.
- **Baseline settings.** Baselines use the default Python standard-library implementations at maximum preset. Other compressors (zstd, brotli, xz with tuned parameters) were not tested.

## 8. Future work

1. Replace the zlib back end with a purpose-built entropy coder for the literal stream, token identifiers, and gap values.
2. Add predictive pre-filters (delta, row-aware prediction) so that numeric and image structure can feed the tokenizer.
3. Improve parsing with cost-aware dynamic programming instead of greedy longest-match, and allow nested tokens (as in grammar-based compression).
4. Extend position patterns beyond exact periodicity, for example approximate periodicity with exception lists, and arithmetic progressions with a bounded number of exceptions.

5. Evaluate on standard corpora and real image datasets, with proper statistics and timing, and add block-wise processing for bounded memory.
6. Compare against zstd and brotli.

## 9. Conclusion

SPCA shows that compressing the structure of token positions is effective when repeated sequences recur in strictly regular patterns, where it can be smaller than zlib by a factor of 2 to 4 and occasionally smaller than LZMA. On realistic text, code, records, and executables, the same approach does not outperform standard compressors, and a cost-based selector falls back to zlib. The results indicate that the idea is best seen as a complement to existing methods for highly regular data, and that stronger entropy coding, better parsing, and domain-aware pre-filters are the prerequisites for any broader advantage.

## Reproducibility

The implementation (`spca.py`), benchmark script (`bench.py`), and raw per-file results (`results.json`) are provided with this report. Fixed random seeds are used for the synthetic data, apart from the random-byte file.

## References


- Burrows, M., and Wheeler, D. J. (1994). A block-sorting lossless data compression algorithm. Digital Equipment Corporation Systems Research Center, Research Report 124.
- Deutsch, P. (1996). DEFLATE Compressed Data Format Specification version 1.3. RFC 1951.
- Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. Proceedings of the IRE, 40(9), 1098-1101.
- Larsson, N. J., and Moffat, A. (2000). Off-line dictionary-based compression. Proceedings of the IEEE, 88(11), 1722-1732.
- Nevill-Manning, C. G., and Witten, I. H. (1997). Identifying hierarchical structure in sequences: a linear-time algorithm. Journal of Artificial Intelligence Research, 7, 67-82.
- Ziv, J., and Lempel, A. (1977). A universal algorithm for sequential data compression. IEEE Transactions on Information Theory, 23(3), 337-343.

- Ziv, J., and Lempel, A. (1978). Compression of individual sequences via variable-rate coding. IEEE Transactions on Information Theory, 24(5), 530-536.